

Métodos Formales para Pruebas

¿Por qué son Necesarias las Pruebas?

Docente: Alejandro Bartra

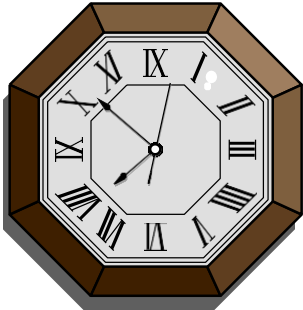
01

¿POR QUÉ SON NECESARIAS LAS
PRUEBAS?

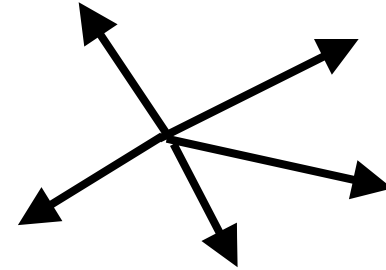
• CONTEXTO DE LAS PRUEBAS



La economía mundial es cada vez más dependiente del software.



Los negocios demandan mayor productividad y CALIDAD en menos tiempo.



Las aplicaciones crecen en tamaño, complejidad y distribución.

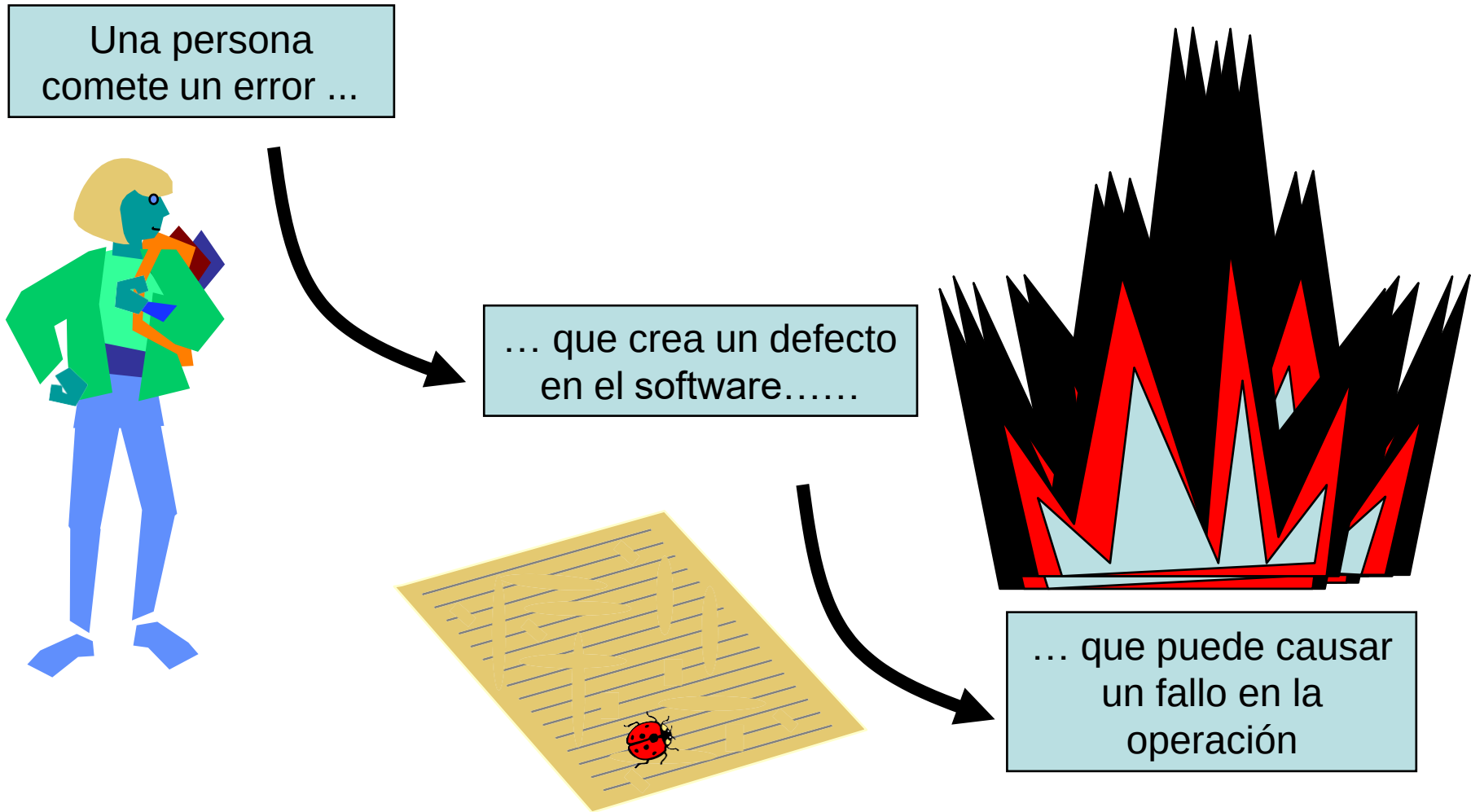


Las pruebas y las revisiones mejoran la CALIDAD del software

- **¿Qué es un Error?: Una acción humana que produce un resultado incorrecto**
- **¿Qué es un Defecto?: Una manifestación de un error en software**
 - También conocido como un defecto o «bug» (bicho, insecto).
 - De ser ejecutado, un defecto puede causar un fallo.
- **¿Qué es un Fallo?: Una desviación del software de su entrega esperada o servicio.**
 - Defecto encontrado

El fallo es un acontecimiento; el defecto es un estado del software, causado por un error.

Error - Defecto - Fallo



Fiabilidad versus Defectos

- **Fiabilidad: la probabilidad que el software no causará el fracaso del sistema durante un tiempo especificado en condiciones especificadas**
 - ¿Puede un sistema estar sin defecto? (cero defectos, la primera vez todo ok) 
 - ¿Puede un sistema de software ser confiable, pero todavía tener defectos? 
 - ¿Una aplicación de software "sin defecto" es siempre confiable? 

- **CAUSAS DE LOS DEFECTOS EN EL SOFTWARE**

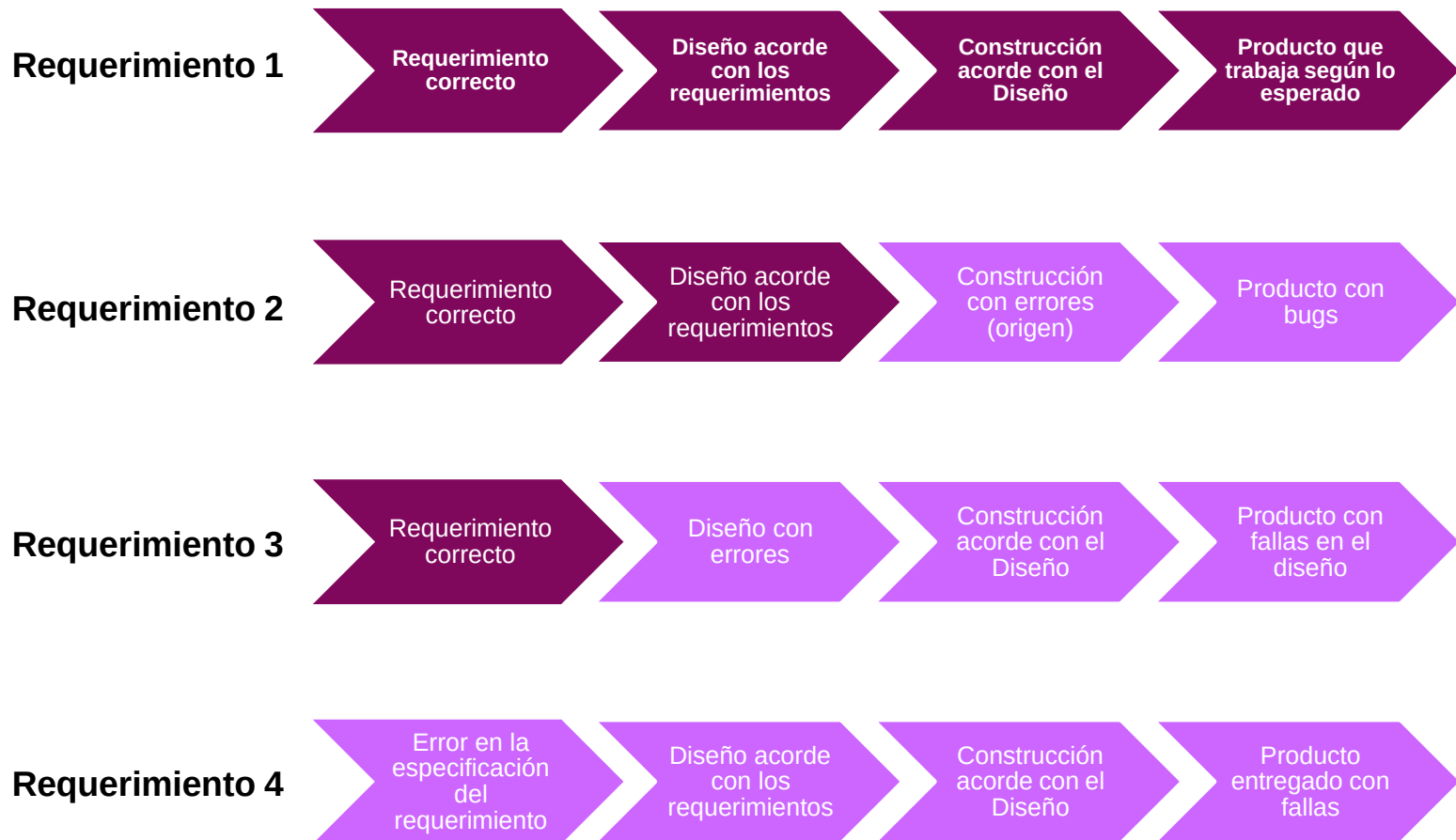
- Error humano
 - Uso incorrecto del software
 - Diseño o construcción del software.
 - Causas de los errores humanos:
 - Plazos, demandas excesivas debido a la complejidad, distracciones, etc.
 - Complejidad técnica del software.
- Condiciones ambientales
 - Cambios en las condiciones ambientales.
 - Causas de condiciones ambientales negativas/adversas:
 - Radiación, campos electromagnéticos, polución, fallo de discos duros, fluctuaciones en el suministro de energía eléctrica.

- **LOS DEFECTOS Y FALLOS PROVIENEN DE:**

- Errores en especificación, diseño e implementación del software.
- Errores en el uso del sistema.
- Condiciones ambientales.
- Daño intencional.
- Consecuencia de errores tempranos, defectos y fallas.

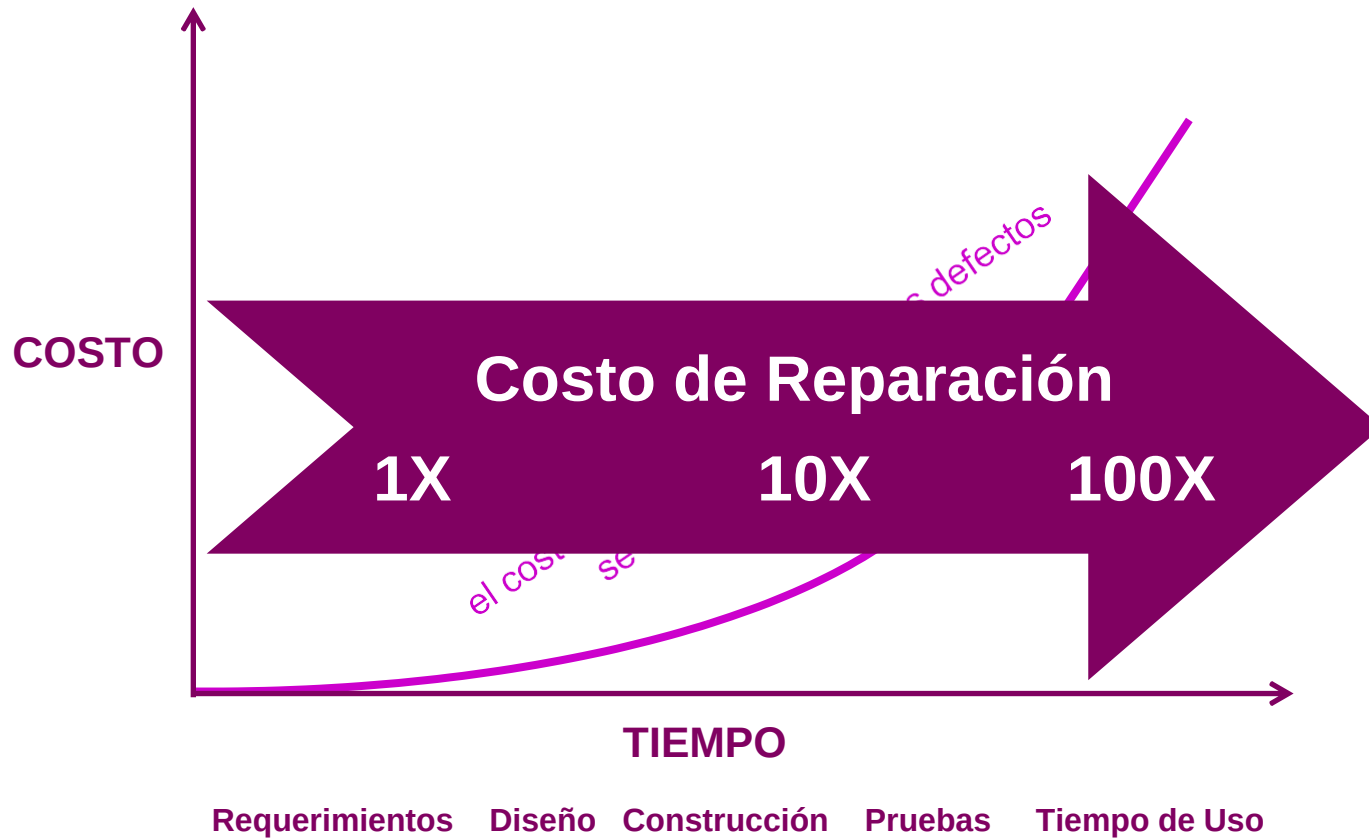
01 ¿POR QUÉ SON NECESARIAS LAS PRUEBAS?

- ¿CUÁNDO APARECEN LOS DEFECTOS?



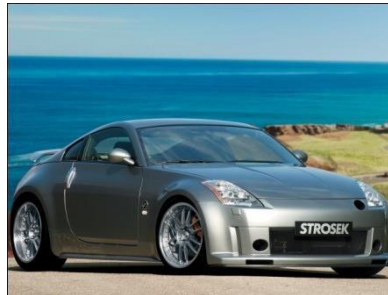
01 ¿POR QUÉ SON NECESARIAS LAS PRUEBAS?

- COSTO DE LOS DEFECTOS



01 ¿POR QUÉ SON NECESARIAS LAS PRUEBAS?

- CALIDAD



delaware

- **CALIDAD**

Grado en que un conjunto de características inherentes cumple con los requisitos

ISO
9000:
2000

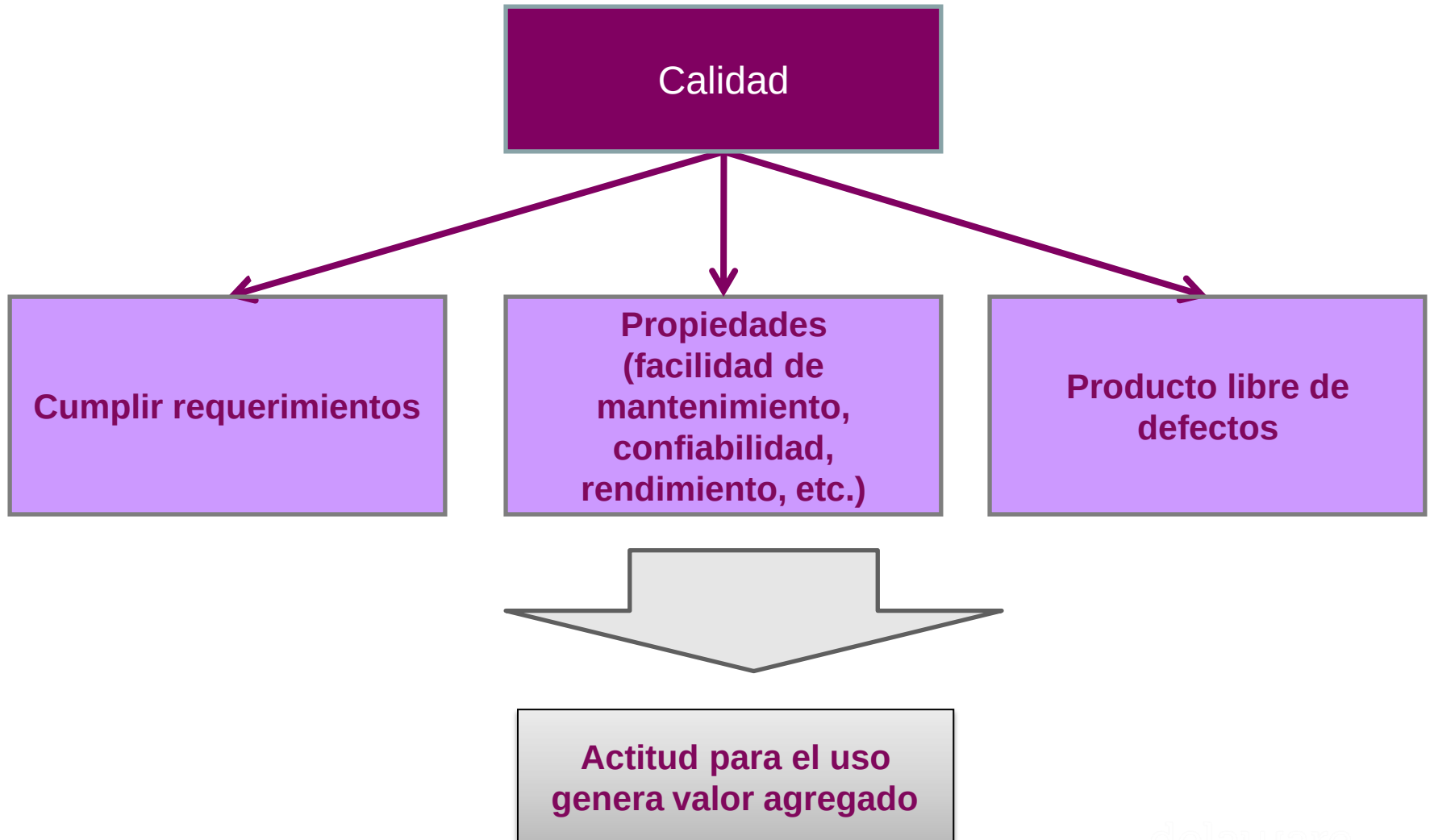
El grado en que un sistema, componente o proceso cumple (1) requerimientos especificados y (2) necesidades o expectativas del cliente o usuario.

IEEE
610.12

Totalidad de características de una entidad (actividad, producto, persona, organización) que le confieren la aptitud para satisfacer las necesidades explícitas e implícitas de sus clientes

ISO
8402

- CALIDAD

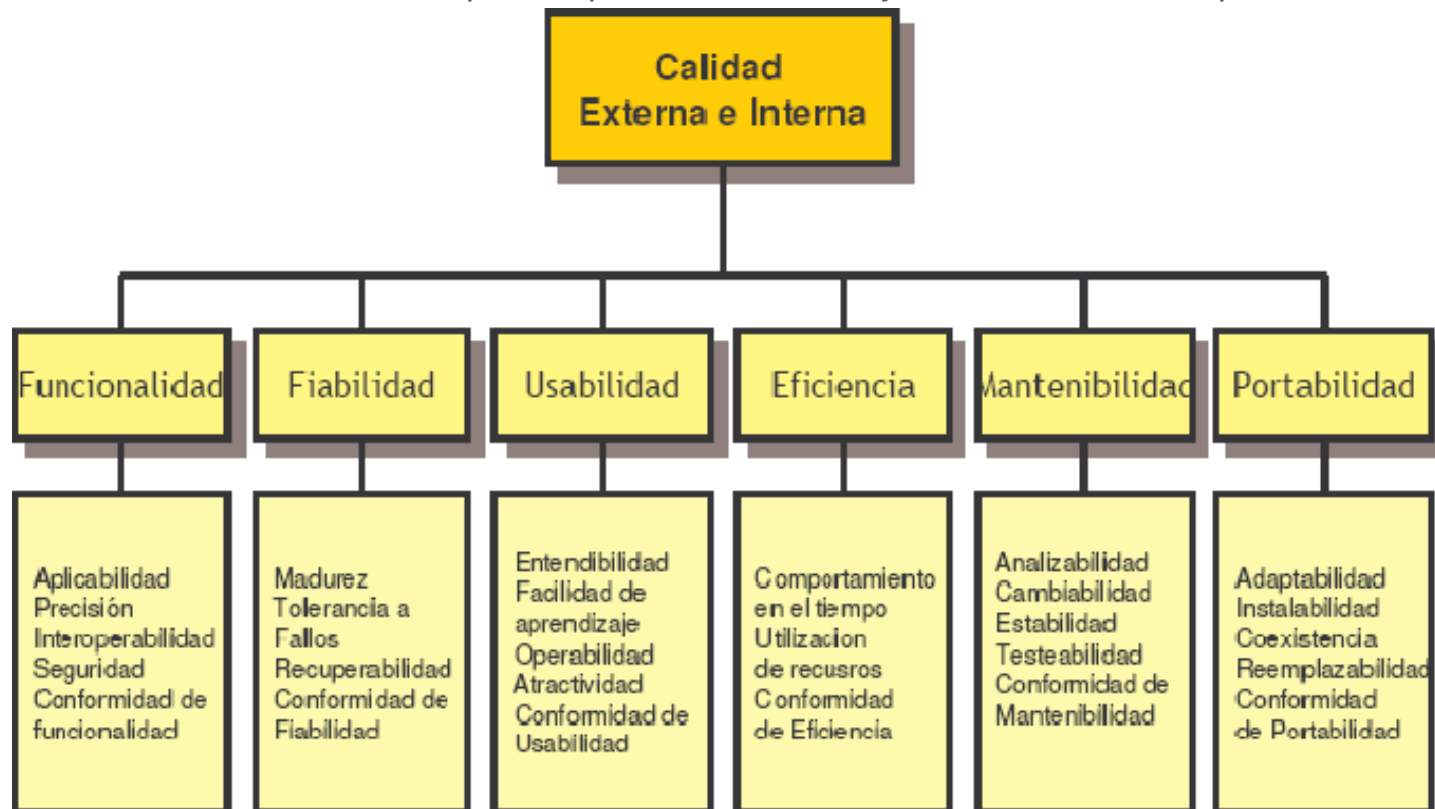


- **PRUEBAS Y CALIDAD**

- Las pruebas ayudan a medir la calidad del software en términos de:
 - cantidad de defectos encontrados,
 - pruebas ejecutadas y
 - cobertura lograda con las pruebas.
- La calidad se puede medir a través de características funcionales (ejemplo: imprimir un reporte correctamente) y no funcionales (ejemplo, impresión rápida del reporte) cubiertas por el software (ISO 9126).

• PRUEBAS Y CALIDAD (ISO 9126)

- Algunos atributos de la calidad de un producto software se influyen mutuamente.
- Debido a esta interdependencia y en función del objeto de prueba, los atributos deberán ser caracterizados por una prioridad a efectos de ser tomada en cuenta. Por ejemplo eficiencia versus portabilidad.
- Se realizarán distintos tipos de pruebas con el objeto de medir cada tipo de atributo.



¿Por qué son importantes las Pruebas?

- Porque el software es probable que tenga fallas
- Para aprender acerca de la fiabilidad del software
- Para llenar el tiempo entre la entrega del software y la fecha de lanzamiento
- Para demostrar que el software no tiene fallos
- Porque la prueba está incluida en el plan del proyecto
- Debido a las fallas que pueden ser muy costosas
- Para evitar ser demandado por los clientes
- Permanecer en el negocio



- **No es una actividad secundaria:**

- 30-40% del esfuerzo de desarrollo
- En aplicaciones críticas (p.ej. control de vuelo, reactores nucleares),
- ¡de 3 a 5 veces más que el resto de pasos juntos de la ingeniería del software!
- El coste aproximado de los errores del software (bugs) para la economía americana es el equivalente al 0,6% de su PIB, unos 60.000 millones de dólares

- **Costos de Calidad:**

- Según Joseph Juran existe la siguiente clasificación de los costos de la calidad:
 - Costos de Prevención
 - Costos de Evaluación
 - Costos por Fallas Internas
 - Costos por Fallas Externas



- **Costos de Prevención:**

- Son los costos de todas las actividades específicamente diseñados para prevenir fallas de calidad en productos o servicios
 - Por ejemplo:
 - Revisión de nuevos productos
 - Planeación de la calidad (manuales, procedimientos, etc.)
 - Evaluación de capacidad de proveedores
 - Esfuerzos de mejora a través de trabajo en equipo
 - Proyectos de mejora continua
 - Educación y entrenamiento en calidad..... etc.



- **Costos de Evaluación**

- Son los costos asociados con las actividades de medir, evaluar y auditar los productos o servicios para asegurar su conformidad con los estándares de calidad y requerimientos de desempeño.

Por ejemplo:

- Inspecciones con el proveedor y en recibo
- Pruebas e inspecciones en proceso y al producto terminado
- Auditorias al producto, proceso o servicio
- Calibración de equipos de prueba y medición
- Costos de materiales de prueba



- **Costos de Falla Interna:**

- Son los costos resultantes de productos o servicios no conformes a los requerimientos o necesidades del cliente, antes del embarque del producto o la realización del servicio.

Por ejemplo:

- Desperdicio
- Re-trabajos
- Re-inspección y repetición de pruebas
- Revisión de materiales no conformes
- Reducción de precio por calidad reducida

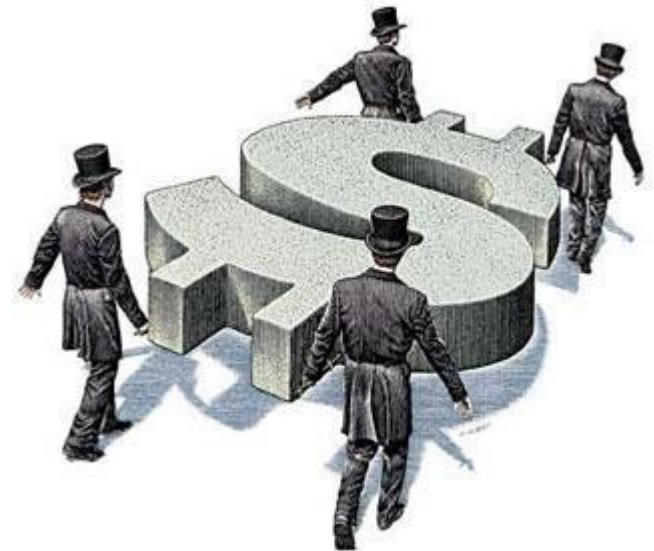


- **Costos de Falla Externa:**

- Son los costos resultantes de productos o servicios no conformes a los requerimientos o necesidades del cliente, después de la entrega del producto o durante y después de la realización del servicio.

Por ejemplo:

- Proceso de quejas y reclamaciones
- Devoluciones del cliente
- Garantías
- Campañas por productos defectivos



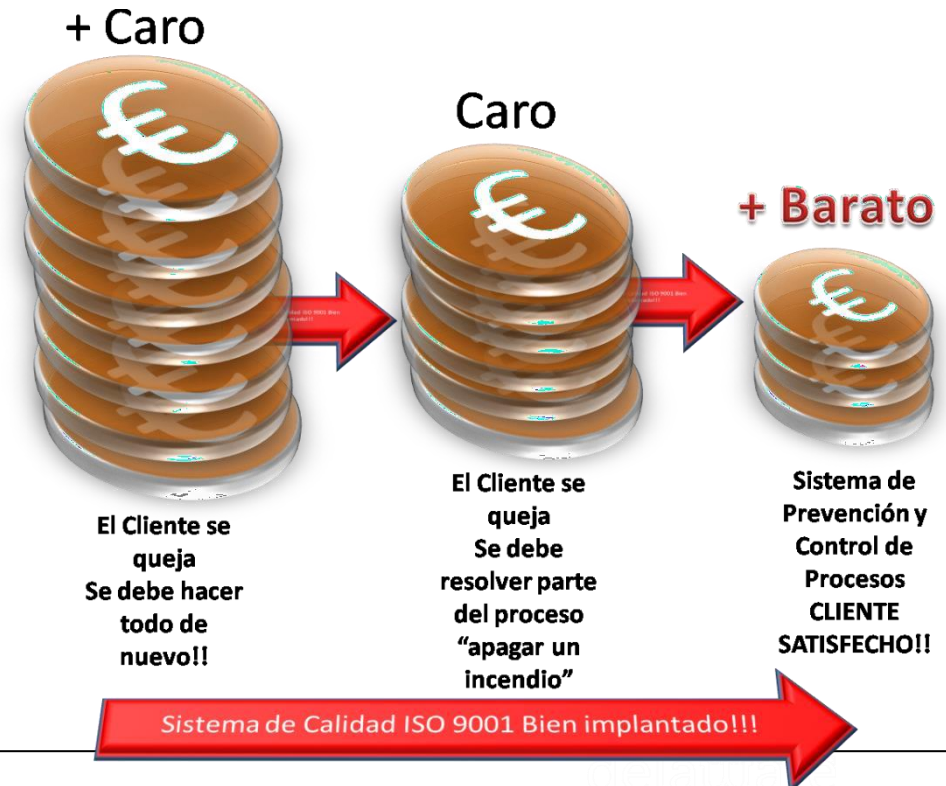
01 ¿POR QUÉ SON NECESARIAS LAS PRUEBAS?

- **Costos Totales de Calidad :**

- Es la suma de los costos de prevención, evaluación, falla interna y falla externa.
- Los sistemas contables en general no son capaces de identificar estos costos.
- Es muy difícil ir al detalle del costo de calidad.

- La Distribución típica de costos directos en las empresas actuales es el

siguiente:Prevención 5%
Evaluación 30%
Fallas internas / externas 65%.



Métodos Formales para Pruebas

Las Pruebas de Software y sus Principios Generales

Docente: Alejandro Bartra

¿QUÉ SON LAS PRUEBAS DE SOFTWARE?

- *DEFINICIÓN:*
 - Testing como proceso:
 - Proceso, con una serie de actividades involucradas.
 - Incurre en todas las actividades del ciclo de vida del software.
 - Tiene técnicas estáticas y dinámicas.
 - Que obedece a una planificación con actividades antes y después de la ejecución de las pruebas como: actividades de ejecución, control, reporte del avance y estado de las pruebas; y cierre de las mismas.
 - Que tiene una preparación, con la elección del tipo de pruebas a realizar, las condiciones de las mismas y los casos de prueba a ejecutar.
 - Que está sometido a evaluación, en la que verificamos los resultados y comprobamos que el software bajo prueba cumple con los criterios de éxito establecidos.
 - En el que se evalúa los productos software y productos de trabajo relacionados.
 - Y con objetivos definidos:
 - Determinar que (los productos software) satisfacen los requerimientos especificados.
 - Determinar que (los productos software) cumplen su propósito.
 - Detectar defectos.

- *OBJETIVOS DE LAS PRUEBAS*

- Determinar que el producto software satisface los requerimientos especificados.
 - Confirma que los productos trabajados apropiadamente reflejan los requerimientos especificados. En otras palabras, la **verificación** asegura que “se construye el producto correctamente”.
 - Confirmación, a través de la provisión de objetivos evidenciables, que los requerimientos especificados han sido cumplidos a cabalidad.
- Determinar que el producto de software cumple su propósito.
 - Confirma que el producto, como condición rutinaria, cumplirá a cabalidad con su uso propuesto. En otras palabras, la **validación** asegura que “se construye el producto correcto”.
- Detectar Defectos.
 - Para descubrir fallas o defectos en el software donde su comportamiento no está en conformidad con su especificación.

- *DEFINICIONES: TEST CASE / TEST BASE*

- Test Case (Caso de Prueba)

- La definición de un caso de prueba incluye la siguiente información (IEEE Std. 610):

- Precondiciones
 - Conjunto de valores de entrada
 - Conjunto de resultados esperados
 - Forma en la cual se debe ejecutar el caso de prueba y verificación de resultados
 - Post condiciones

- Test Base o Test Basis (Base de Pruebas / Fundamentos de pruebas)

- Conjunto de documentos que definen los requisitos de un componente o sistema, utilizado como referencia para el desarrollo de casos de prueba.

- **DEFINICIONES: CODE / DEBUGGING / SOFTWARE DEVELOPMENT**

- Code o Source Code (código fuente)
 - Programa de computadora escrito en un lenguaje de programación que puede ser leído por una persona.
- Debugging (Depuración)
 - Localización y corrección de defectos en el código fuente.
- Software Development (Desarrollo de Software)
 - Es un proceso / secuencia de actividades cuyo objetivo es desarrollar un sistema basado en un computador (ordenador).

- **DEFINICIONES: REQUIREMENT / REVIEW**

- Requirement (Requisito)
 - Un requisito describe un atributo funcional o no funcional deseado o considerado obligatorio.
- Review (Revisión)
 - Evaluación de un producto o estado de un proyecto con el objeto de detectar discrepancias con respecto a los resultados esperados (planificados) y para recomendar mejoras (IEEE Std 1028).

PRINCIPIOS GENERALES DE LAS PRUEBAS

- *PRINCIPIO 1: LAS PRUEBAS MUESTRAN LA PRESENCIA DE DEFECTOS*

- La prueba puede mostrar que los defectos están presentes, pero no pueden probar que no existen defectos.
- La prueba reduce la probabilidad de existencia de defectos no descubiertos, pero eso no demuestra su ausencia; incluso cuando los defectos no son encontrados.
- Las pruebas reducen la probabilidad de la presencia de defectos que permanecen sin ser detectados. La ausencia de fallos no demuestran que el producto software es correcto.
- El mismo proceso de pruebas puede contener errores.
- Las condiciones de las pruebas pueden ser inapropiadas para detectar errores.

- *PRINCIPIO 2: LA PRUEBA EXHAUSTIVA ES IMPOSIBLE*

- Probar todo (todas las combinaciones de entrada y precondiciones) no es factible, excepto para casos triviales. En lugar de test exhaustivos, se utilizan técnicas basadas en riesgos y prioridades para enfocar los esfuerzos de las pruebas.
- Pruebas exhaustivas ("exhaustive testing").
 - Enfoque del proceso de pruebas donde el conjunto de pruebas abarca todas las combinaciones de valores de entrada y precondiciones.
- Explosión de casos de prueba ("test case explosión").
 - Define el incremento exponencial de esfuerzo y costo en el caso de pruebas exhaustivas.
- Muestras para probar ("sample test").
 - La prueba incluye solamente a un subconjunto (generado de forma sistemática o aleatoria) de todos los posibles valores de entrada.
 - En condiciones normales, se utilizan generalmente muestras para probar. Probar todas las combinaciones posibles de entradas y precondiciones sólo es económicamente viable en casos triviales.

Pruebas Exhaustivas? NECESARIAS LAS PRUEBAS?

- *¿Qué es una prueba exhaustiva?*

- cuando todos los probadores se han agotado
- cuando todas las pruebas previstas se han ejecutado
- el ejercicio de todas las combinaciones de los insumos y las condiciones previas



- **¿Cuánto tiempo toma las pruebas exhaustivas?**

- tiempo infinito
- no mucho tiempo
- cantidad de tiempo práctico



- *PRINCIPIO 3: PROBAR LO MÁS TEMPRANAMENTE POSIBLE*

- Las actividades de pruebas deben comenzar tan pronto como sea posible en el ciclo de vida del desarrollo del software o sistema y deben ser enfocadas sobre objetivos definidos.
- La corrección de un defecto es menos costosa en la medida en la cual su detección se realiza en fases más tempranas del proceso software.
- Los conceptos y especificaciones pueden ser probados.
- Los defectos detectados en la fase de concepción son corregidos con menor esfuerzo y costo.
- El proceso de pruebas implica preparación de una prueba también consume tiempo más que sólo la ejecución de pruebas.
- Las actividades de pruebas pueden ser preparadas antes de que el desarrollo se haya completado.
- Las actividades de pruebas (incluidas las revisiones) deben ser ejecutadas en paralelo a la especificación y diseño software.
- Se obtiene una máxima rentabilidad cuando los errores son corregidos antes de la implementación.

01 ¿POR QUÉ SON NECESARIAS LAS PRUEBAS?

- nunca es suficiente.
- cuando usted ha hecho lo que estaba previsto.
- cuando el cliente / usuario es feliz.
- cuando se ha demostrado que el sistema funciona correctamente.
- cuando se tiene la certeza de que el sistema funciona correctamente.
- depende de los riesgos para su sistema.



- *PRINCIPIO 4: AGRUPAMIENTO DE DEFECTOS*

- Un número pequeño de módulos contienen la mayoría de los defectos descubiertos durante la prueba, o muestran la mayoría de los fallos operacionales.
 - ¡Encuentre un defecto y encontrará más defectos "cerca"!
 - Los defectos aparecen agrupados.
 - Cuando se detecta un defecto es conveniente investigar el mismo módulo en el que ha sido detectado.
- Los probadores ("testers") deben ser flexibles.
- Habiendo sido detectado un defecto es conveniente volver a considerar el rumbo de las pruebas siguientes.
- La identificación de un defecto puede ser investigada con un mayor grado de detalle. Por ejemplo, realizando pruebas adicionales o modificando pruebas existentes.

¿Cuánto probar?

- Depende del **RIESGO**
 - Riesgo de perder las fallas importantes.
 - Riesgo de incurrir en los costes de fallos.
 - Riesgo de liberación de software no probado o pruebas de baja calidad.
 - Riesgo de perder credibilidad y la participación en el mercado.
 - Riesgo de perder una ventana de mercado.
 - Riesgo de un exceso de pruebas, las pruebas ineficaces.

Tan poco tiempo, mucho para probar...

- *Tiempo de prueba siempre será limitado*
 - Utilice el **RIESGO** para determinar lo siguiente:
 - ¿Qué probar primero?
 - ¿Qué pruebas son más importantes?
 - ¿Qué no se debe probar (en este momento)?
- } ¿Dónde poner énfasis?
- Utilice el **RIESGO** para :
 - Asignar el tiempo disponible para probar y priorizar las pruebas.

- *PRINCIPIO 5: LA PARADOJA DEL PESTICIDA*

- Si el mismo test se repite una y otra vez, eventualmente el mismo grupo de casos de prueba ya no encontrará nuevos errores. Para vencer esta “paradoja del pesticida”, los casos de prueba necesitan ser regularmente revisados y estudiados, y se necesitan escribir nuevos casos de prueba para probar diferentes partes del software o sistema a fin de encontrar más defectos potenciales.
- Repetir pruebas en las mismas condiciones no es efectivo.
- Cada caso de prueba debe contar con una combinación única de parámetros de entrada para un objeto de pruebas particular, de lo contrario no se podrá obtener información adicional.
- Si se ejecutan las mismas pruebas de forma reiterada no se podrán encontrar nuevos defectos.
- Las pruebas deben ser revisadas/modificadas regularmente para los distintos módulos (código)
- Es necesario repetir una prueba tras una modificación del código (corrección de defectos, nueva funcionalidad).
- La automatización de pruebas puede resultar conveniente si un conjunto de casos de prueba se deben ejecutar frecuentemente.

- *PRINCIPIO 6: LA PRUEBA DEPENDE DEL CONTEXTO*

- La prueba se realiza de manera diferente en diferentes contextos. Por ejemplo, el software de seguridad crítico se prueba de forma diferente a la de un sitio de comercio electrónico.
- Objetos de prueba diferentes son probados de forma diferente.
 - El firmware del motor de un coche requiere pruebas diferentes respecto de aquellas definidas para una aplicación de e-commerce.
- Entorno de pruebas vs. entorno de producción.
 - Las pruebas tienen lugar en un entorno distinto del entorno de producción. El entorno de pruebas debería ser similar al entorno de producción.
 - Siempre habrá diferencias entre el entorno de pruebas y el entorno de producción. Estas diferencias introducen incertidumbre con respecto a las conclusiones que se pudieran obtener tras las pruebas.

01 ¿POR QUÉ SON NECESARIAS LAS PRUEBAS?

03 PRINCIPIOS GENERALES DE LAS PRUEBAS

- *PRINCIPIO 7: LA FALACIA DE LA AUSENCIA DE ERRORES*
 - Encontrar y solventar defectos no ayuda si el sistema construido está inutilizado y no satisface las necesidades y expectativas de los usuarios.
 - Un proceso de pruebas adecuado detectará los fallos más importantes.
 - En la mayoría de los casos el proceso de pruebas no encontrará todos los defectos del sistema (ver Principio 2), pero los defectos más importantes deberían ser detectados.
 - Esto, en sí, no prueba la calidad del sistema software.
 - La funcionalidad del software puede no satisfacer las necesidades y expectativas de los usuarios.
 - No se puede introducir la calidad a través de las pruebas; la calidad tiene que construirse desde el principio.

Métodos Formales para Pruebas

Procesos Fundamentales de las Pruebas

Docente: Alejandro Bartra

PROCESOS FUNDAMENTALES DE

LAS PRUEBAS

- DEFINICIONES: TEST STRATEGY / EXIT CRITERIA / TEST CONDITION

- *Test strategy (Estrategia de pruebas)*

- (1) Descripción sucinta de los niveles de pruebas a realizar y las pruebas asociadas a una organización o programa (uno o más proyectos).
- (2) Conformidad con el enfoque global, los esfuerzos requeridos para las pruebas se distribuyen entre los diferentes objetos de prueba y objetivos de las pruebas:
 - elección del método de prueba.
 - cómo y cuándo se ejecutarán las actividades asociadas a las pruebas.
 - cuándo se debe detener el proceso de pruebas (exit criteria)

- *Exit criteria (Criterio de salida)*

- Conjunto de condiciones genéricas y específicas, acordadas con los interesados, para permitir que un proceso sea considerado formalmente finalizado. El propósito de los criterios de salida es evitar que una tarea se considere finalizada habiendo partes importantes de la misma sin completar. Los criterios de salida son utilizados como fuente para elaborar informes y planificar la suspensión de las pruebas.

- *Test condition (Condición de Pruebas)*

- Un objeto o evento de un componente o sistema que debe ser verificado por uno o más casos de prueba. Ejemplos: una función, transacción, característica, atributo de calidad o elemento estructural.

- DEFINICIONES: TEST SUITE / TEST PROCEDURE / TEST EXECUTION

- *Test Suite (Suite de Pruebas)*

- Conjunto de casos de prueba para un componente o sistema, donde la pos condición de un caso de prueba es utilizada como precondition para el caso de prueba siguiente.

- *Test Procedure Specification (especificación del procedimiento de pruebas)/Test scenario (escenario de pruebas)*

- Documento en el cual se especifica una secuencia de acciones para la ejecución de una prueba. También es conocido como test script o manual test script [IEEE Std 829].

- *Test execution (Ejecución de pruebas)*

- Es el proceso de ejecutar una prueba en un sistema o componente para producir el resultado actual.

- DEFINICIONES: TEST LOG / REGRESSION TESTING / CONFIRMATION TESTING /RE-TESTING
 - *Test log (registro de pruebas) / Test Report (Informe de Pruebas)*
 - Registro cronológico de los detalles relevantes ocurridos durante la ejecución de las pruebas.
 - *Confirmation Testing (Pruebas de Confirmación) / Re-Testing (Reiteración de Pruebas)*
 - Pruebas que ejecutan casos de prueba que fallaron la última vez que fueron ejecutados, con el objetivo de verificar el éxito de las acciones correctivas.
 - *Regression Testing (Pruebas de Regresión)*
 - Pruebas realizadas a un programa previamente probado considerando sus modificaciones para asegurar que no se introduzcan nuevos defectos en las áreas no modificadas del software, como resultado de los cambios realizados. Son realizadas cuando el software o entorno han sido modificados.

- DEFINICIONES: TEST PLAN / TEST DATA / INPUT DATA

- *Test plan (Plan de pruebas)*

- Documento que describe el alcance, el enfoque, recursos y planificación de las actividades previstas en el proceso de pruebas. Identifica, entre otros objetos de prueba, las características a ser probadas, las tareas de pruebas, quién realizará cada tarea, grado de independencia del tester, el ambiente de pruebas, las técnicas de diseño de pruebas, los criterios de salida (exit criteria), y la justificación racional de la elección. [IEEE 829]

- *Test data (Datos de prueba)*

- Datos que existen en el sistema antes de que una prueba sea ejecutada, y que afecta o es afectado por los componentes o sistema sujeto a pruebas.

- *Input data (Datos de entrada)*

- Variable que es leída por un componente del sistema.

- DEFINICIONES: TEST COVERAGE/ TEST ORACLE

- *Test coverage (Cobertura de pruebas)*

- El grado, expresado en porcentaje, en el cual un elemento específico ha sido ejecutado por una suite de pruebas.

- *Test oracle (Oráculo de pruebas)*

- Una fuente que permite determinar los resultados esperados para compararlos con los resultados actuales del software sujeto a pruebas.

Un oráculo puede ser un software existente (para un benchmark), un manual de usuario, un individuo con conocimiento especializado, pero de ninguna manera debe ser el código. [Adrion]

- EL PROCESO DE PRUEBAS EN EL PROCESO DE DESARROLLO DESOFTWARE

- Dependiendo del enfoque seleccionado el proceso de pruebas será realizado en varios puntos del proceso de desarrollo.
 - Las pruebas constituyen un proceso en sí mismas.
- El proceso de pruebas está determinado por las siguientes actividades:
 - Planificación y control
 - Análisis y diseño de pruebas
 - Implementación y ejecución
 - Evaluación del criterio de finalización de pruebas y generación de informes de pruebas.
 - Cierre de pruebas.

• EL PROCESO DE PRUEBAS DURANTE EL PROCESO DE DESARROLLO DE SOFTWARE

- Las actividades del Proceso de Pruebas generalmente se desarrollan de manera secuencial, pero en algunos proyectos, toman lugar de manera concurrente o incluso se repiten.
- El Proceso de Pruebas es más que la ejecución de pruebas
- Cada fase del Proceso de Pruebas tiene lugar de forma concurrente con las fases del proceso de Desarrollo de software.



- PLANIFICACIÓN Y CONTROL

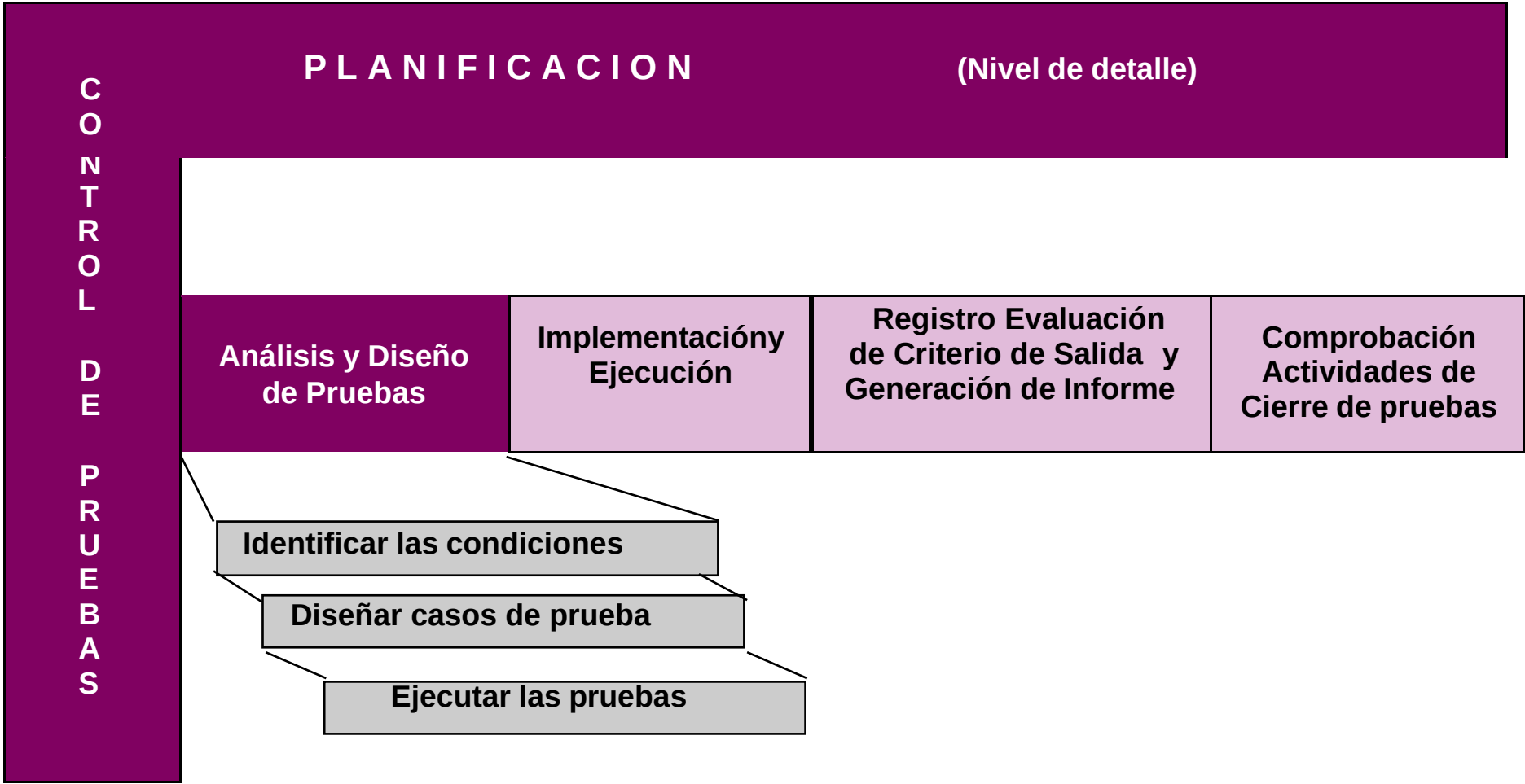
- *Planificación*

- Determinar el alcance y riesgos.
 - Identificar los objetivos de las pruebas y el criterio de finalización de pruebas.
 - Determinar el *test approach* (enfoque de pruebas)
 - técnicas de pruebas, elementos a probar, cobertura, identificación de las interfaces entre los equipos involucrados en las pruebas, y *testware* (artefactos producidos durante el proceso de pruebas).
 - Implementar las políticas de pruebas y/o la estrategia de pruebas
 - Planificar las tareas de: análisis y diseño; implementación y ejecución de pruebas; y evaluación.
 - Adquirir / obtener y programar recursos requeridos por las pruebas:
 - personal, entorno de pruebas, herramientas, presupuesto de pruebas.
 - Definir el *exit criteria* (criterio de salida)

- PLANIFICACIÓN Y CONTROL

- *Control*

- El control de pruebas es una actividad continua que influye en la planificación de las pruebas. El plan de pruebas puede ser modificado en función de la retroalimentación proporcionada por la actividad de control de pruebas.
- El estado del proceso de pruebas se determina comparando el progreso logrado con respecto al plan de pruebas. Se iniciarán aquellas actividades que se consideraran consecuentemente necesarias.
- Tareas:
 - Medir y analizar los resultados de las revisiones y pruebas.
 - Monitorear el progreso de las pruebas, la cobertura de las pruebas y el cumplimiento del *exit criteria* (criterio de finalización)
 - Proveer información de las pruebas
 - Iniciar medidas correctivas.
 - Tomar decisiones.



- ANÁLISIS Y DISEÑO

- Revisión de los *Test Base* (requisitos, arquitectura del sistema, diseño, interfaces), analizando las especificaciones del software a probar.
- Identificar los *test conditions* (condiciones de prueba) basándose en:
 - el análisis de los objetos de pruebas,
 - sus especificaciones y
 - conocimiento del comportamiento y estructura.
- Diseño de las pruebas
 - Se definen los casos y procedimientos de prueba.
 - Crear casos de prueba lógicos (casos de prueba con datos de prueba sin valores específicos) y establecer un orden de prioridad para los mismos.
 - Los casos de prueba positivos comprueban la funcionalidad, los casos de prueba negativos comprueban situaciones en la cuales se debe realizar un tratamiento a los errores.
- Evaluar la *testability* (característica que define la capacidad que posee un software de ser probado) de los requerimientos y el sistema.
- Diseñar la configuración del ambiente de pruebas e identificar la infraestructura y herramientas.



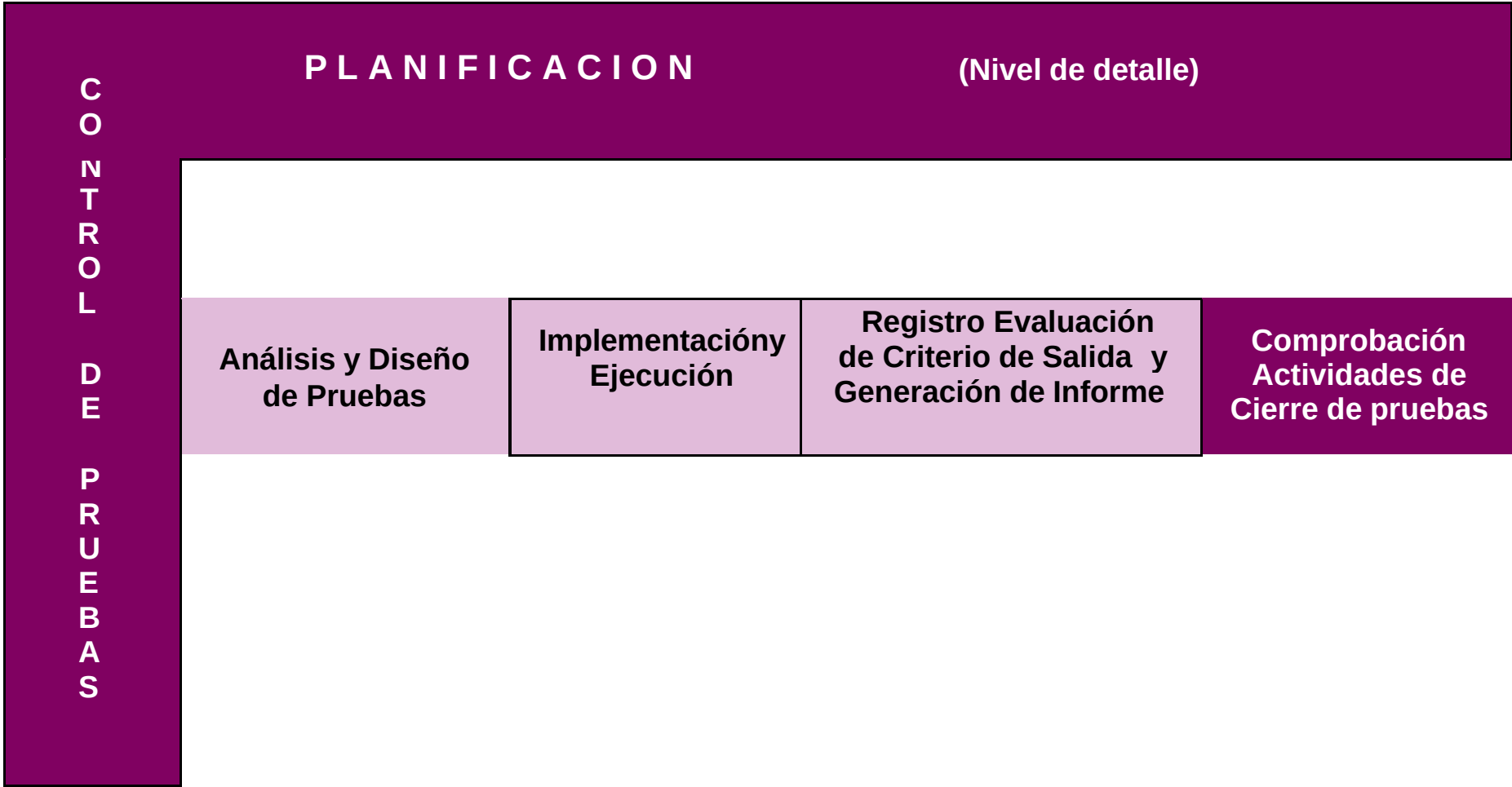
- IMPLEMENTACIÓN Y EJECUCIÓN
- Durante la implementación y ejecución de las pruebas se toman las *test conditions* (condiciones de pruebas) y se elaboran los casos de prueba y el *testware* respectivo. Además, se configura el ambiente de pruebas.
- Implementación
 - Desarrollar y priorizar los casos de prueba.
 - Crear los datos de prueba.
 - Elaborar los procedimientos de prueba.
 - Crear las *test suites* desde los casos de prueba para definir una ejecución de las pruebas eficientes
 - Implementar y verificar el ambiente.

- IMPLEMENTACIÓN Y EJECUCIÓN

- Durante la implementación y ejecución de las pruebas se toman las *test conditions* (condiciones de pruebas) y se elaboran los casos de prueba y el *testware* respectivo. Además, se configura el ambiente de pruebas.
- Ejecución
 - Ejecutar las test suite (suite de pruebas) y los casos de pruebas individuales, siguiendo la secuencia definida en los *test procedures* (procedimientos de pruebas). La ejecución de las pruebas puede ser de forma manual o automática.
 - Registrar el resultado de la ejecución de las pruebas, la identificación y versión del software probado, herramientas de pruebas y *testware*.
 - Comparar los resultados actuales con los resultados esperados.
 - Reportar las discrepancias entre los resultados actuales y esperados como incidentes.
 - Repetir las pruebas para cada incidente.
 - Luego de una corrección se repiten las pruebas con el objeto de asegurar que los defectos han sido corregidos y que la corrección no introduce nuevos defectos.



- EVALUACIÓN DEL CRITERIO DE SALIDA Y GENERACIÓN DE INFORMES
 - Evaluar los resultados de la ejecución de pruebas contra el *exit criteria* (criterio de salida) definido en la planificación de las pruebas.
 - Evaluar si es necesario realizar más actividades de pruebas o si el *exit criteria* (criterio de salida) debe ser cambiado.
 - Generar un *test summary report* (reporte resumen de las pruebas) para los *interesados*



- ACTIVIDADES DE CIERRE

- Comprobar qué entregables planificados han sido entregados.
- Informe de las incidencias cerradas y diferidas.
- Cerrar y archivar los *testware* (artefactos generados en el proceso e pruebas), tales como scripts, el entorno de pruebas y la infraestructura de pruebas.
- Analizar y registrar las lecciones aprendidas para futuros proyectos.
- Otras actividades:
 - Documentar la aceptación del sistema.

PSICOLOGÍA DE LAS PRUEBAS

- ROLES Y RESPONSABILIDADES

- **DESARROLLO**

Implementar requisitos y especificaciones

Implementar el diseño del software

Desarrollar el software

Construye el producto

Localizar y corregir defectos

- **PRUEBAS**

Asegurar que los requisitos y especificaciones del producto sean correctamente implementados

Diseñar las pruebas

Hallar fallos y defectos del software para reportarlos como incidencias

Revisa la calidad del producto durante su construcción

Valida que las acciones correctivas tomadas en base a la incidencias sean exitosas

- ¡La actividad del desarrollador es constructiva!
- ¡La actividad del tester es destructiva!
- ¡Error! ¡Las pruebas también constituyen una actividad constructiva, su propósito es la eliminación de defectos en un producto!

- CARACTERÍSTICAS PERSONALES DEL TESTER

- Curioso, perceptivo, atento a los detalles
 - Con el objeto de comprender los escenarios prácticos del cliente.
 - Con el objeto de poder analizar la estructura de una prueba.
 - Con el objeto de descubrir dónde se pueden manifestar los fallos.
- Escéptico y con actitud crítica.
 - Los objetos de prueba contienen defectos y el objetivo es encontrarlos.
 - No se debe tener temor al hecho de que pueden ser descubiertos defectos serios que tengan un impacto sobre la evolución del proyecto.

- CARACTERÍSTICAS PERSONALES DEL TESTER

- Aptitudes para la comunicación.
 - Necesarias para comunicar malas noticias a los desarrolladores.
 - Necesarias para vencer estados de frustración.
 - Tanto cuestiones técnicas como prácticas, relativas al uso del sistema, deben ser entendidas y comunicadas.
 - Una comunicación positiva puede ayudar a evitar o facilitar situaciones difíciles.
 - Facilitan el establecimiento de una relación de trabajo con los desarrolladores a corto plazo.
- Experiencia.
 - Factores personales influyen en la detección de los errores.
 - La experiencia ayuda a identificar dónde se pueden acumular errores.

- PRUEBAS INDEPENDIENTES

- La separación de las responsabilidades de pruebas apoya y/o promueve la evaluación independiente de los resultados de las pruebas.
- Niveles de independencia
 - Pruebas realizadas por los autores de los objetos de prueba.
 - Pruebas realizadas por otra persona del mismo equipo.
 - Pruebas realizadas por una persona de un grupo organizacional diferente, como un equipo de pruebas independiente.

¿Por qué probar?

- Fomentar la confianza
- Probar que el software es correcto
- Demostrar la conformidad con los requisitos
- Encontrar fallas
- Reducir los costes
- Sistema cumple con las necesidades del usuario
- Evaluar la calidad del software



Métodos Formales para Pruebas

Modelos de Desarrollo de Software

Docente: Alejandro Bartra

06

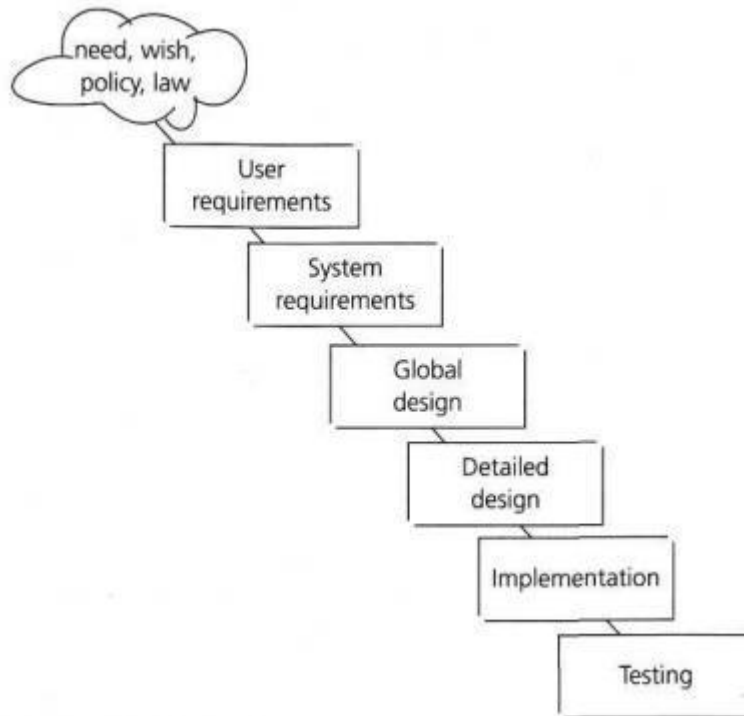
MODELOS DE DESARROLLO DE SOFTWARE

INTRODUCCIÓN

- El proceso de desarrollo adoptado para un proyecto, dependerá de sus objetivos y proyecciones.
- Las pruebas se dan dentro del ciclo de vida del desarrollo de software. El tipo y nivel de la prueba se define según la parte del ciclo de vida en el que se encuentra el desarrollo del software.
- Es importante conocer el modelo de ciclo de vida adoptado por el proyecto para poder definir correctamente la estrategia de prueba del mismo.
- En todos los ciclos de vida, una parte de las pruebas está enfocada a la “**Verificación**” y otra parte a la “**Validación**”.

MODELO CASCADA

- El modelo de cascada fue uno de los primeros modelos que se diseñó. Tiene una línea de tiempo natural, donde las tareas se ejecutan de manera secuencial.



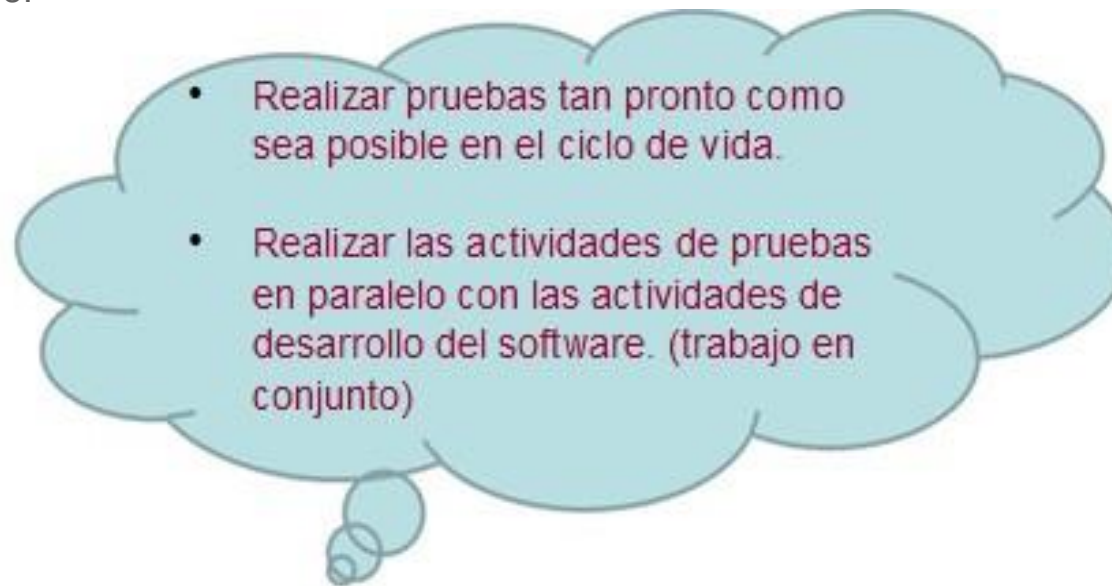
Deficiencias:

Los errores se encontraban en etapas muy avanzadas del proyecto (generaba mayores costos y tiempo)

Es un modelo muy lineal y en consecuencia no puede soportar muchas iteraciones (cambios o variaciones en el ciclo de vida).

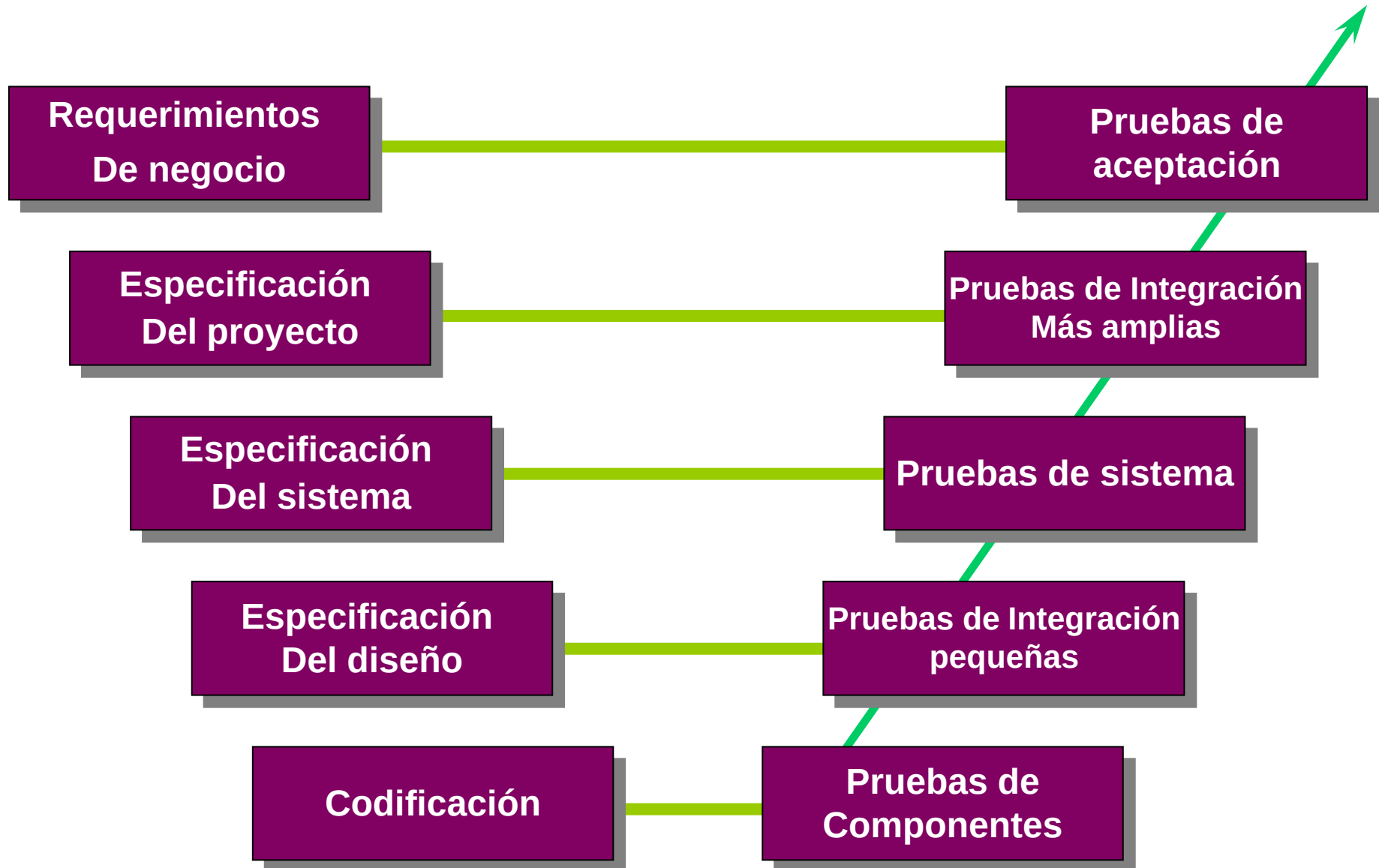
V-MODEL

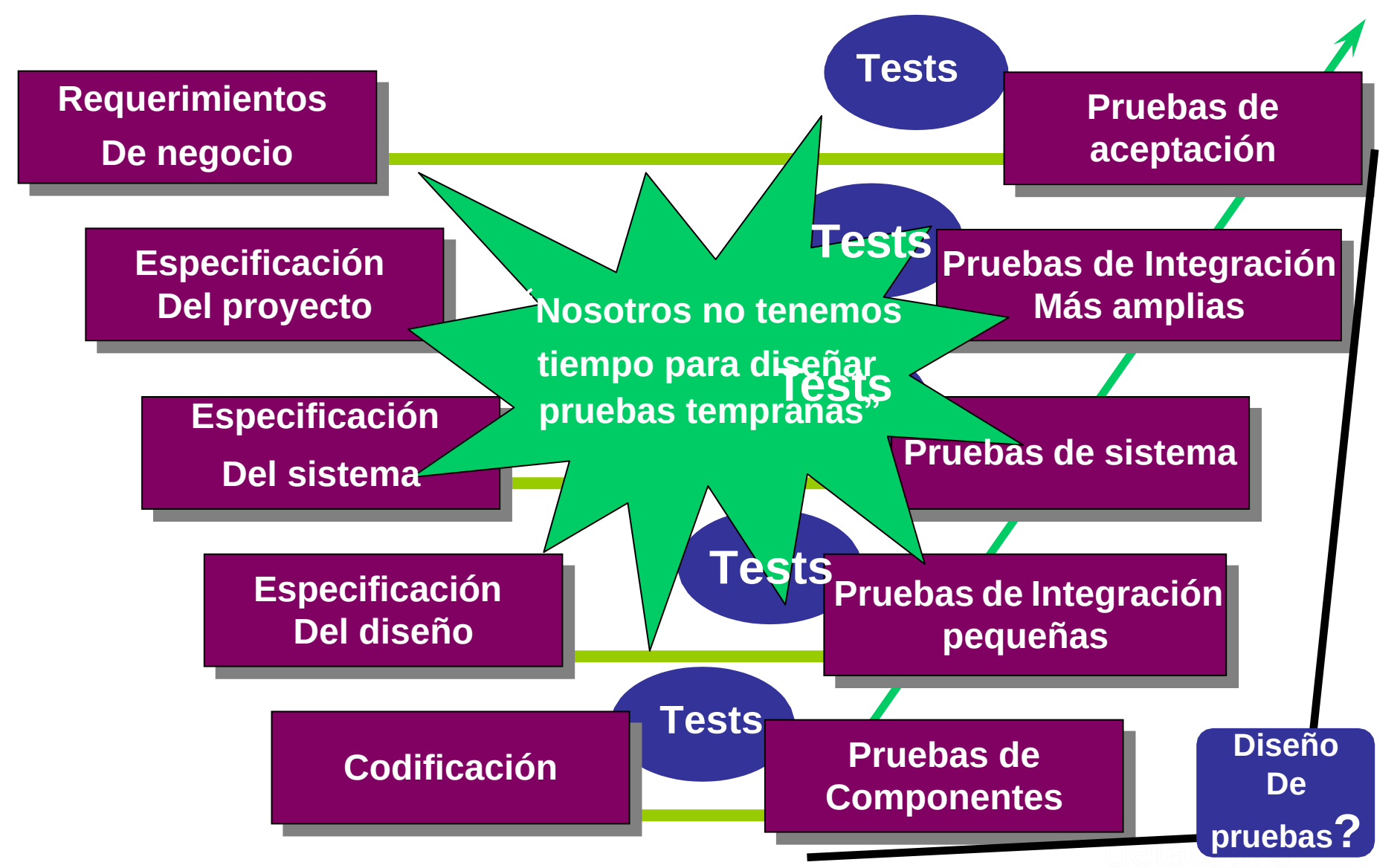
- El V-modelo se desarrolló para abordar algunos de los problemas experimentados utilizando el método de cascada tradicional.
- El V-model es un modelo que ilustra cómo las actividades de las pruebas (verificación y validación) pueden ser integradas en cada fase del ciclo de vida. Dentro del V-model, las pruebas de **validación** se llevan a cabo especialmente durante las primeras etapas, por ejemplo revisar los requisitos del usuario, y al final del ciclo de vida, por ejemplo, durante las pruebas de aceptación del usuario.

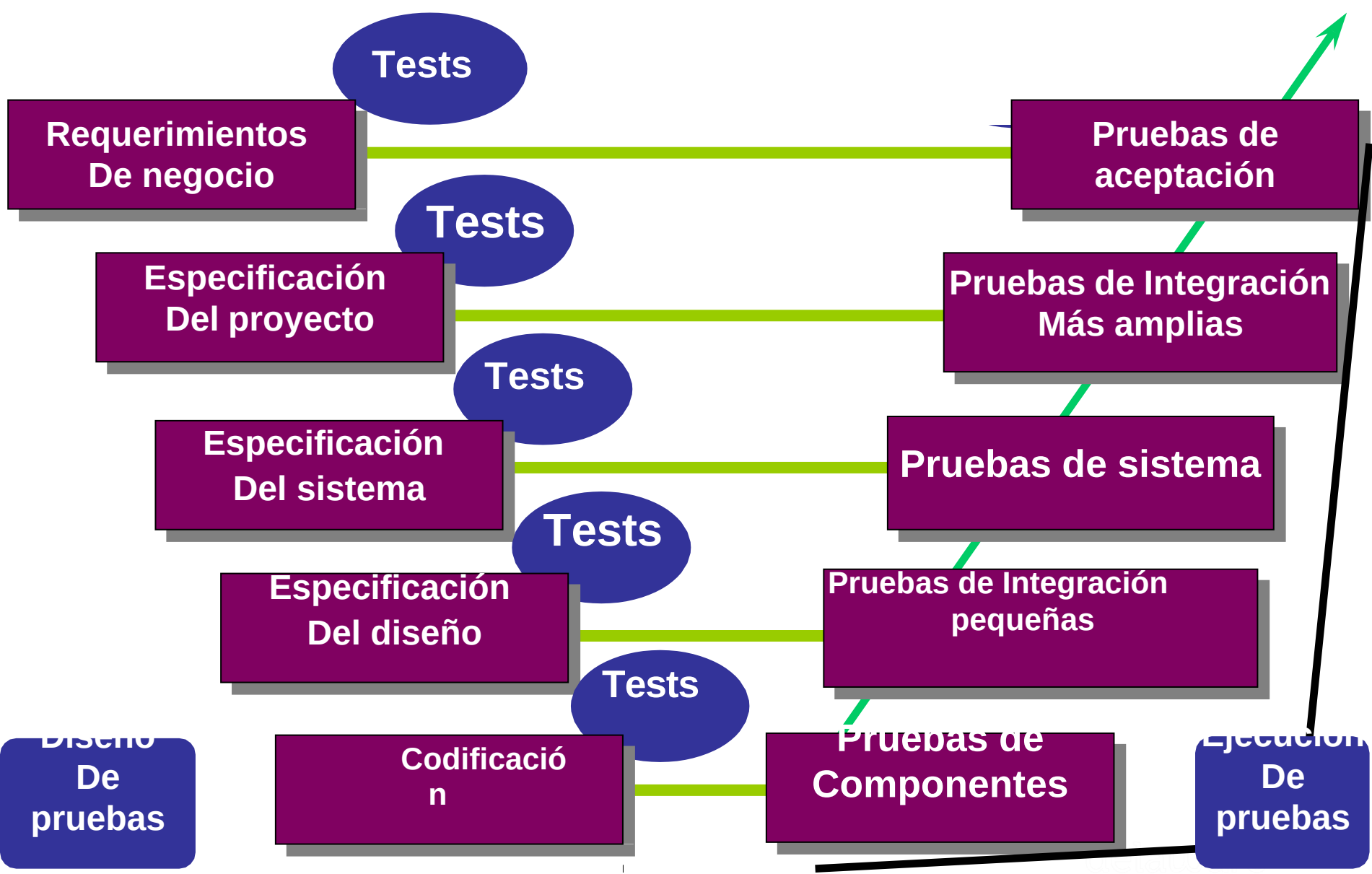


V MODEL

- Los cuatro niveles de prueba utilizados son los siguientes:
 - **Pruebas de componentes:** busca defectos y verifica el funcionamiento del componente de software (por ejemplo, módulos, programas, objetos, clases) que se comprueban por separado.
 - **Pruebas de integración:** pruebas de interfaces entre los componentes, las interacciones a diferentes partes de un sistema, por ejemplo: el sistema operativo, sistema de archivos y hardware o interfaces entre los sistemas.
 - **Pruebas del sistema:** concernientes al comportamiento de todo el sistema / producto tal como está definido por el alcance del proyecto. El objetivo principal de las pruebas del sistema es la verificación de los requerimientos especificados.
 - **Pruebas de aceptación:** las pruebas de validación con respecto a las necesidades del usuario, requerimientos y procesos empresariales, para determinar si procede o no aceptar el sistema.

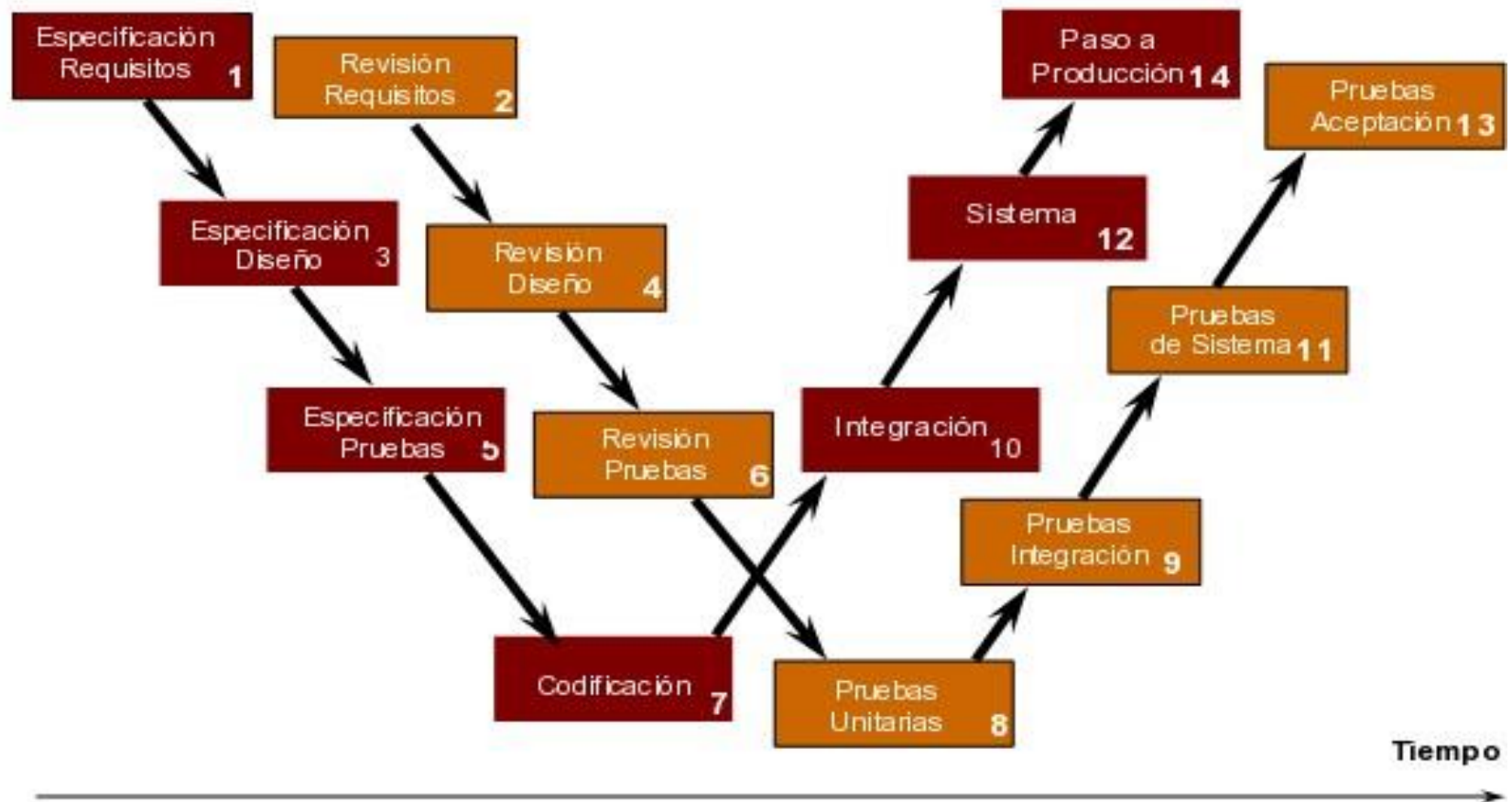






V-MODEL

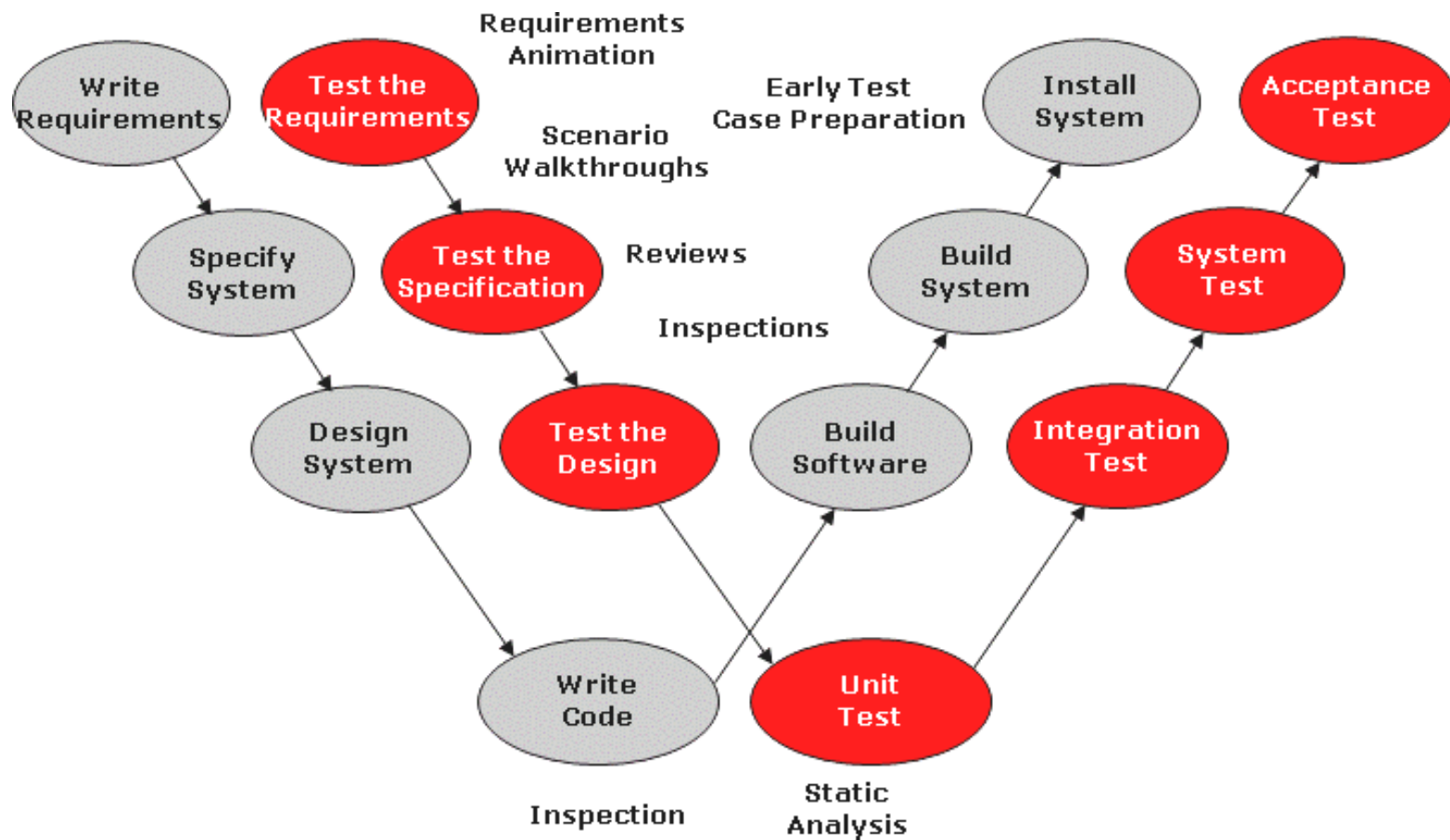
Gestionar las pruebas desde el comienzo del proyecto



- CMMI
- ISO / IEC 12207

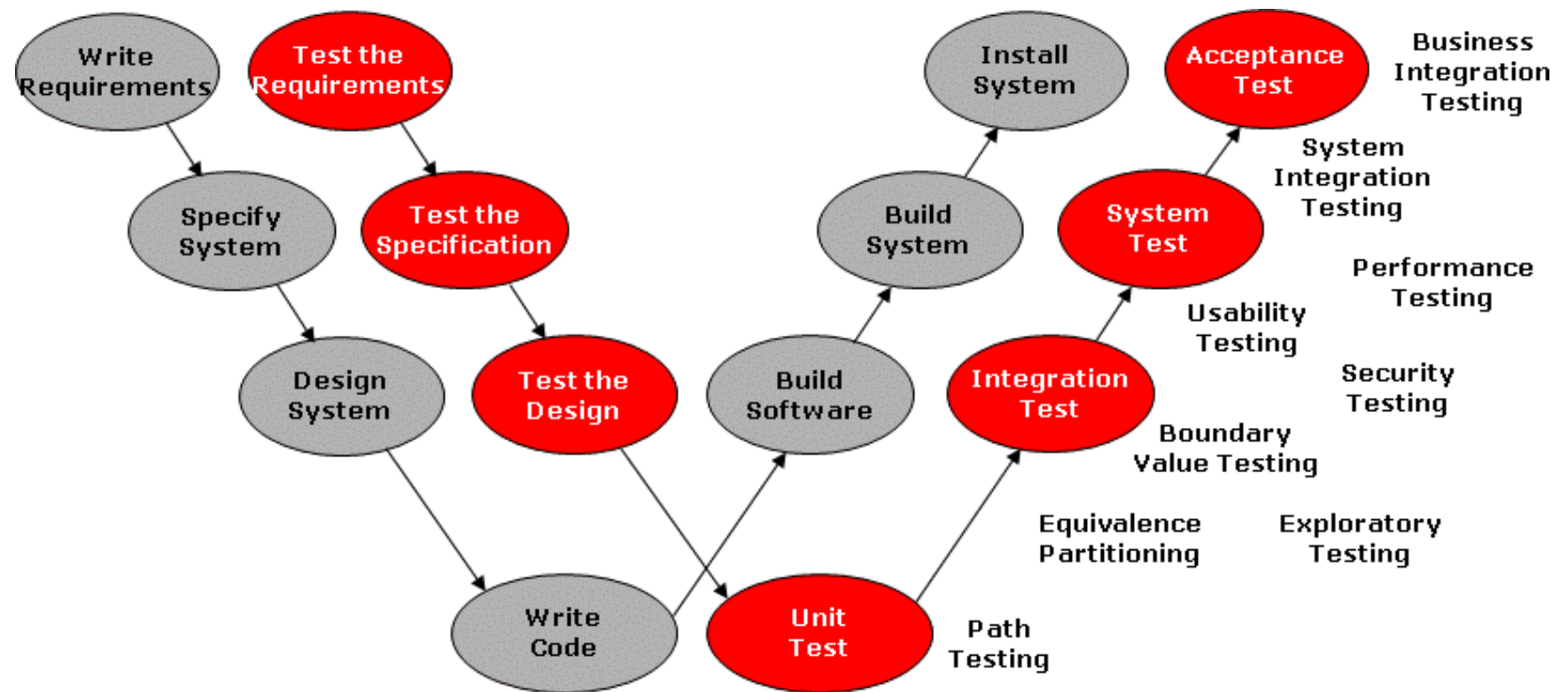
W-MODEL

The W-Model and static test techniques



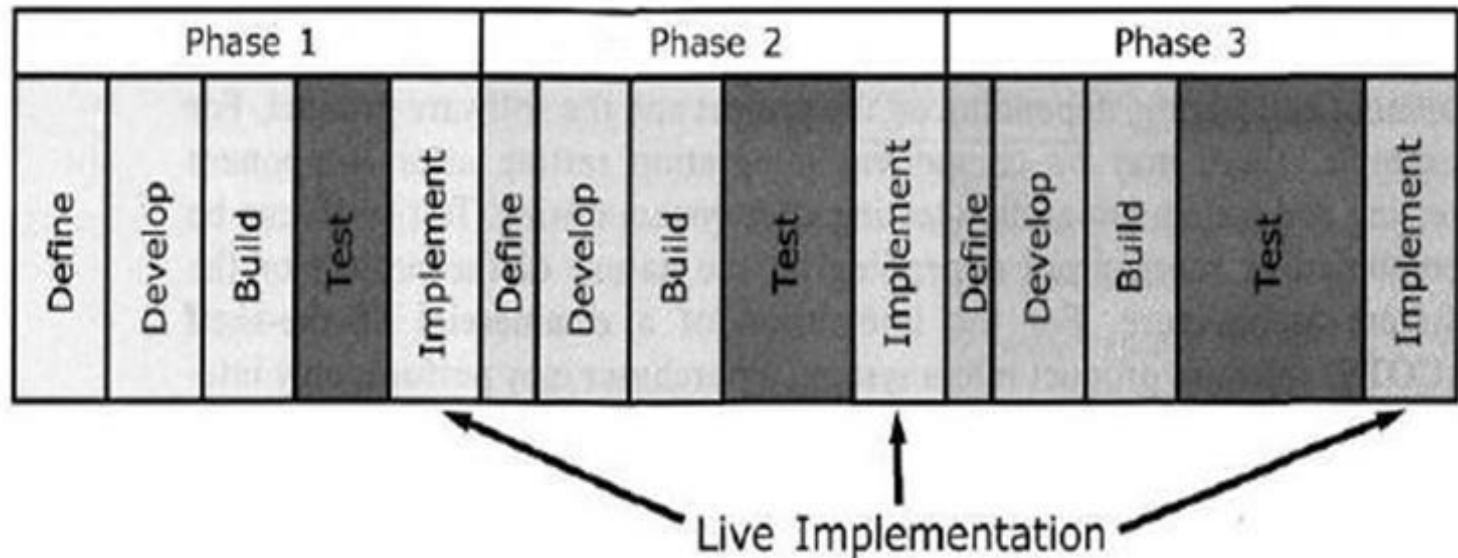
W-MODEL

The W-Model and dynamic test techniques



CICLOS DE VIDA ITERATIVO

- En lugar de trabajar en una gran línea de tiempo, se puede dividir un proyecto en pequeñas ciclos de vida, de manera que se puedan aplicar las revisiones y pruebas a cada iteración de principio a fin.



(*)Pruebas de integración y regresión

CICLOS DE VIDA ITERATIVO

- Ejemplos de modelos de desarrollo iterativo o incremental son: Prototipos, Desarrollo Rápido de Aplicaciones (RAD), Rational Unified Process (RUP) y Desarrollo ágil (Scrum).
- RAD: funciones/componentes son desarrolladas en paralelo como si se trataran de mini proyectos.

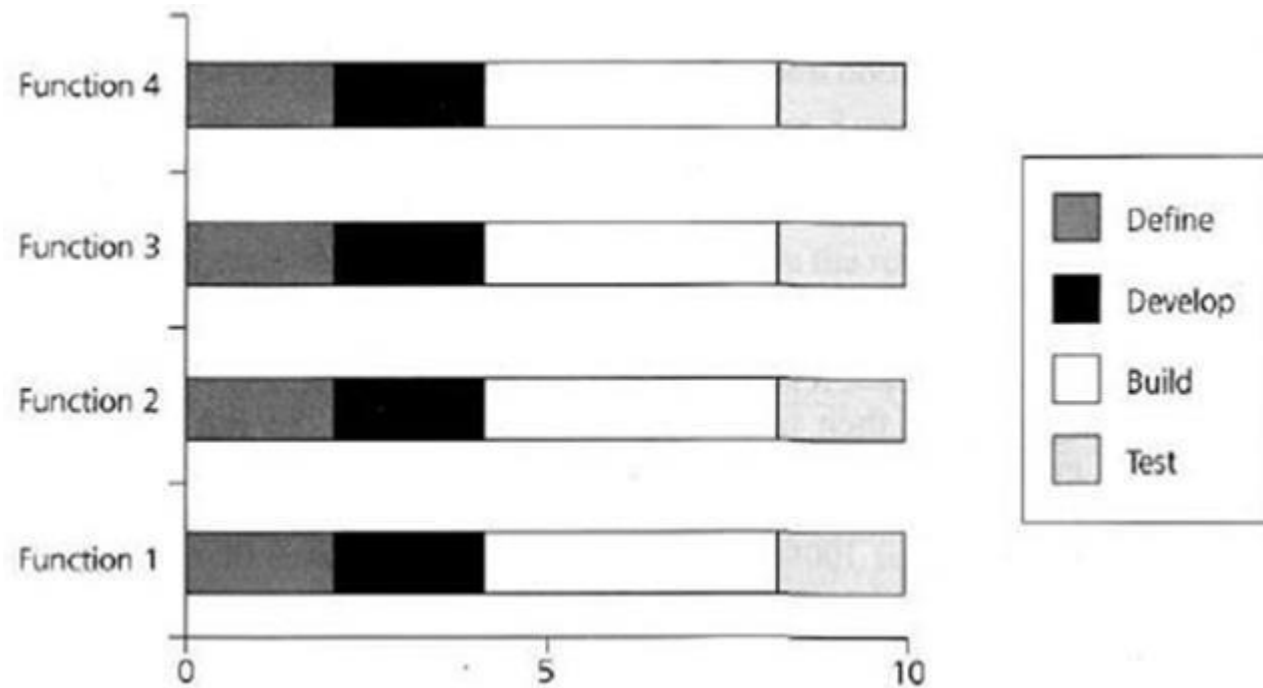


FIGURE 2.4 RAD model

CICLOS DE VIDA ITERATIVO: Características

- Desarrollo en paralelo de componentes y funciones.
- Rápidos resultados iniciales (El cliente puede ver algo del proyecto desde un principio)
- Rápida respuesta ante los cambios en los requerimientos de los clientes.
- Retroalimentación por parte de los clientes.

01 ¿POR QUÉ SON NECESARIAS LAS PRUEBAS?

PRUEBAS DENTRO DE UN MODELO DE CICLO DE VIDA

- Por cada actividad de desarrollo existe una actividad de prueba correspondiente.
- Cada nivel de prueba tiene un objetivo de prueba específico para ese nivel.
- El análisis y diseño de las pruebas para un nivel de prueba debe comenzar durante las actividades de desarrollo correspondientes.
- El Tester deben participar en la revisión de los documentos tan pronto como estén disponibles en el ciclo de desarrollo.